

BİLGİSAYAR MÜHENDİSLİĞİ YÜKSEK LİSANS BSM585 ÖNERİ SİSTEMLERİ

ÖDEV 1

Collaborative Filtering: User-Based ve Item-Based Yöntemlerin Karşılaştırmalı Analizi

1. Amaç ve Genel Bakış

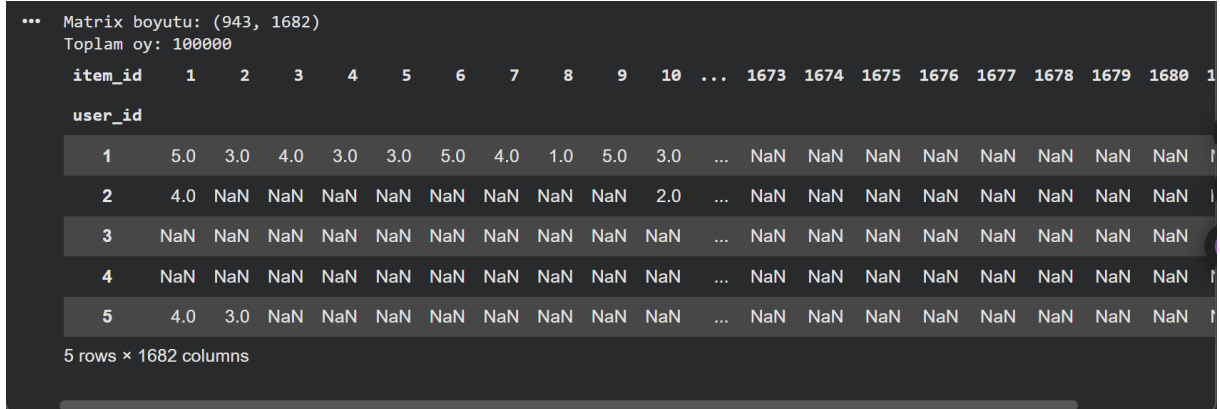
Bu ödev kapsamında MovieLens 100K veri seti üzerinde iki farklı Collaborative Filtering (CF) yöntemi sıfırdan geliştirilecektir: User-Based CF (PCC benzerliği) ve Item-Based CF (ACS benzerliği). Farklı komşu boyutları (k) test edilerek MAE metrikleri karşılaştırılacak; veri yoğunluğunun (sparsity) model performansına etkisi deneysel olarak incelenecektir.

Ödevin temel hedefleri:

- PCC ve ACS benzerlik metriklerini sıfırdan, doğru biçimde implemente etmek
- Leave-One-Out Cross-Validation uygulamak ve MAE hesaplamak
- $k = \{10, 25, 50, 100\}$ değerleri için karşılaştırmalı deney yürütmek ve çalışma sürelerini ölçmek
- %20 veri silme senaryosuyla sparsity etkisini analiz etmek
- Bulguları grafik, tablo ve akademik düzeyde yorumla raporlamak

2. Veri Seti

MovieLens 100K veri setini <https://grouplens.org/datasets/movielens/> adresinden indirin. 943 kullanıcı, 1682 film, yaklaşık 100.000 oy (1–5 tam sayı) içermektedir.



```
... Matrix boyutu: (943, 1682)
Toplam oy: 100000
item_id    1    2    3    4    5    6    7    8    9   10   ...  1673  1674  1675  1676  1677  1678  1679  1680  1
user_id
1         5.0  3.0  4.0  3.0  3.0  5.0  4.0  1.0  5.0  3.0  ...  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  1
2         4.0  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  2.0  ...  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  1
3         NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  ...  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  1
4         NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  ...  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  1
5         4.0  3.0  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  ...  NaN  NaN  NaN  NaN  NaN  NaN  NaN  NaN  1
5 rows x 1682 columns
```

2.1. Zorunlu Veri Ön İşleme

• Mükerrer (user_id, item_id) çiftlerinin tespiti ve nasıl ele alındığının açıklanması: Mükerrer kayıt, aynı verilerin birden fazla kez kaydedilmesi durumudur, bu projedeki kayıtlar ise aynı kullanıcı aynı filme birden fazla puan vermesi durumu olarak ele alınmaktadır. Mükerrer değerler pivot tablo oluştururken sıkıntıya sebep olur bu yüzden kaldırılması gerekir. Ön işlemede mükerrer durumda olan oylamalarda ilk oy bırakılarak diğer oylar silinecek şekilde kod yazılmıştır.

- Eksik değer (NaN) sayısının raporlanması
- Sparsity oranının hesaplanması: $sparsity = 1 - (|R| / (|U| \times |I|))$
- Bu sparsity değerinin CF yöntemleri üzerindeki etkisinin yorumlanması (en az 3 cümle)

MovieLens 100K veri seti %93,70 sparsity oranına sahiptir. Bu yüksek sparsity, CF yöntemlerinin zorlandığı bir durum olarak ele alınmaktadır çünkü kullanıcıların çoğu filmi oylamamıştır. User-based CF, benzer kullanıcı bulmakta zorlanabilir; Item-based CF ise film benzerliklerini daha stabil hesaplayabilir.

2. Veri Ön İşleme

```
# Mükerrer: tekrar, aynı kullanıcı aynı filme birden fazla puan mı vermiş?
duplicates = df.duplicated(subset=['user_id', 'item_id']).sum()
print(f"Mükerrer (user_id, item_id) çifti: {duplicates}") # 0 olmalı yani tekrar oylama işlemi olmamalı
# Aynı kullanıcı aynı filmi birden fazla kes oyladıysa SİL
if duplicates > 0:
    df = df.drop_duplicates(subset=['user_id', 'item_id']) # ilk oy tutulur diğerleri silinir
    ratings_matrix = df.pivot_table(index='user_id', columns='item_id', values='rating')

# Boş değerlerin kontrolü
nan_count = ratings_matrix.isna().sum().sum()
total_possible = ratings_matrix.shape[0] * ratings_matrix.shape[1]
sparsity = 1 - (df.shape[0] / total_possible)
print(f"NaN sayısı: {nan_count}")
print(f"Sparsity: {sparsity:.4f} ({sparsity*100:.2f}%)")

... Mükerrer (user_id, item_id) çifti: 0
NaN sayısı: 1486126
Sparsity: 0.9370 (%93.70)
```

3. Kısım 1: User-Based Collaborative Filtering (PCC)

3.1. Benzerlik Fonksiyonu: Pearson Korelasyon Katsayısı (PCC)

PCC, aktif kullanıcı a ile diğer kullanıcılar arasındaki benzerliği, ortak değerlendirilen ürünler üzerinden ölçer.

Zorunlu Fonksiyon İmzası

```
def compute_pcc(ratings_matrix, user_a, user_u):
    """
    Aktif kullanıcı user_a ile kullanıcı user_u arasındaki PCC değerini hesaplar.
    Params:
        ratings_matrix : pd.DataFrame - satırlar kullanıcı, sütunlar film, değerler oy (NaN = oylanmamış)
        user_a : int - aktif kullanıcı ID'si
        user_u : int - karşılaştırılan kullanıcı ID'si
    Returns:
        float - PCC [-1,1]; ortak ürün <2 ise 0.0
    """
```

Fonksiyon 5 ana bölümden oluşturulmuştur:

1. İki kullanıcının oylarını alır
2. İkisinin birlikte oyladığı filmleri bulunur
3. Ortalama çıkarılır (bias düzeltilmesi): bazı kullanıcılar her şeye yüksek, bazıları her şeye düşük puan verir. Ortalama çıkarılarak bu kişisel eğilim temizlenir.
4. PCC formülü hesaplanır

- İki kullanıcı da bir filmi ortalamasının **üstünde** oyladıysa → çarpım **pozitif** → benzerlik artar
- Biri üstte, diğeri alttaysa → çarpım **negatif** → benzerlik azalır

$$sim(a, b) = \frac{\sum_{p \in P} (r_{a,p} - \bar{r}_a)(r_{b,p} - \bar{r}_b)}{\sqrt{\sum_{p \in P} (r_{a,p} - \bar{r}_a)^2} \sqrt{\sum_{p \in P} (r_{b,p} - \bar{r}_b)^2}}$$

5. Sıfır Standart Sapma Kontrolü

```
# 5. Sıfır Standart Sapma Kontrolü
if den_a == 0 or den_u == 0:
    return 0.0
return num / (den_a * den_u)
```

Bir kullanıcı tüm filmlere aynı puanı verdiyse (hep 3.0 gibi), standart sapma 0 olur bu da sıfıra bölme hatası → 0.0 döner. Kod bu durumda ZeroDivisionError verir bu durumun önüne geçilmesi için bu kod satırı yazılmıştır.

3.2. Tahmin Fonksiyonu

Benzer kullanıcıların o filme verdiği puanlara bakarak oradan tahmin üretir.

$$p_{a,i} = \bar{r}_a + \frac{\sum_{u=1}^k (r_{u,i} - \bar{r}_u) \times sim(a,u)}{\sum_{u=1}^k sim(a,u)}$$

- ▶ $\bar{r}_a$ = mean rating for user a
- ▶ $u_1, \dots, u_k$ are the k -nearest-neighbors to a
- ▶ $r_{u,i}$ = rating of user u on item i
- ▶ $sim(a,u)$ = Pearson correlation between a and u

Kullanıcı A, Film X için tahmin

- └ A'nın ortalamasını hesapla (Film X hariç)
- └ Film X'i oylayan tüm kullanıcılarla benzerlik hesapla
- └ En benzer K kişiyi seç
- └ Her komşunun Film X'e verdiği sapmaları benzerlikle ağırlıklandır
- └ A'nın ortalaması + ağırlıklı ortalama sapma = TAHMİN

3.3. Değerlendirme: Leave-One-Out CV

Protokol

1. Her kullanıcı sırayla aktif kullanıcı olarak seçilir.
2. Aktif kullanıcının oylarından 5 film rastgele seçilir (random seed = 42).

3. Bu 5 filmden 1 tanesi test ürünü olarak ayrılır; gerçek oy saklanır, veriden geçici olarak çıkarılır.
4. Kalan 4 film ve diğer kullanıcıların verileri eğitim kümesini oluşturur.
5. Tahmin üretilmediğinde (yeterli komşu yoksa) kullanıcı MAE hesabına dahil edilmez; bu durum sayılır ve raporda belirtilir.

MAE

Tahmin edilen değerler (p_i) ile gerçek değer (o_i) arasındaki hata MAE ile hesaplanır. R ise tahmin edilen test verilerinin sayısıdır. Unutulmamalıdır ki, düşük MAE sonuçları daha iyi sonuçları gösterir.

$$MAE = \frac{1}{R} \sum_{i=1}^R |p_i - o_i|$$

Her kullanıcı için ayrı MAE hesaplayınız, ardından tüm kullanıcıları kapsayan global MAE değerini bulunuz.

```
return results

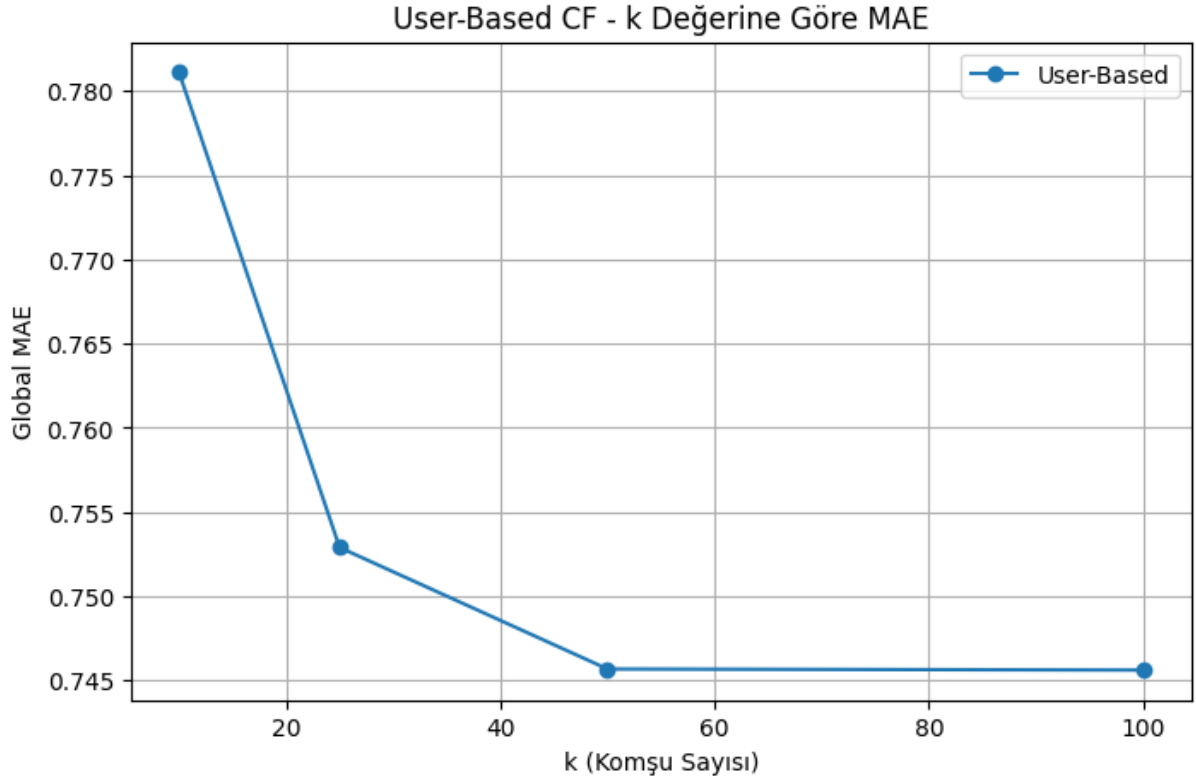
user_results = evaluate_user_based()

... k=10 → MAE: 0.7811, Süre: 172.60s, NoPred: 0
    k=25 → MAE: 0.7529, Süre: 174.49s, NoPred: 0
    k=50 → MAE: 0.7457, Süre: 180.43s, NoPred: 0
    k=100 → MAE: 0.7456, Süre: 187.27s, NoPred: 0
```

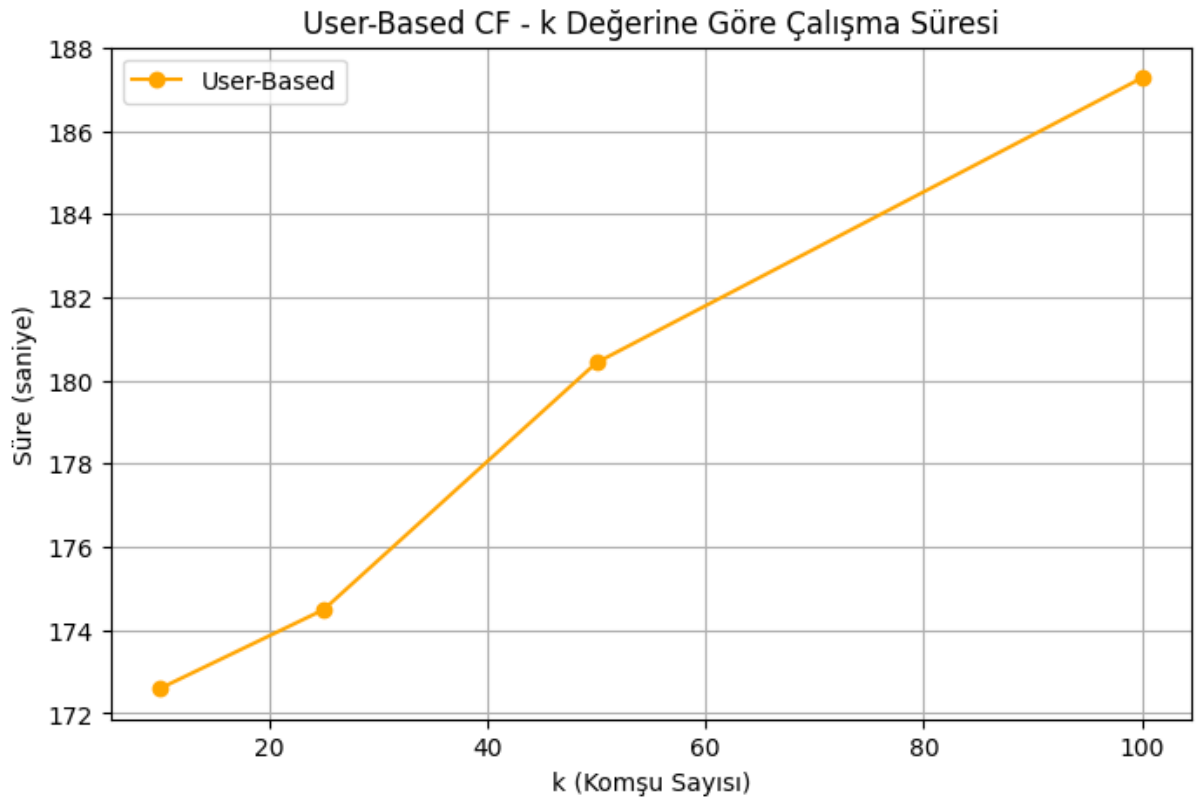
3.4. k Değeri Deneyleri ve Süre Ölçümü

Metrik	k=10	k=25	k=50	k=100
Global MAE	0.7811	0.7529	0.7457	0.7456
Toplam Süre (sn)	172.60	174.49	180.43	187.27
Çalışma Ortamı	Google Colab			

- $k \rightarrow$ MAE değerlerini çizgi grafikte gösterin; en iyi ve en kötü noktaları işaretleyin.



• $k \rightarrow$ Süre değerlerini ayrı bir çizgi grafikte gösterin.



• Eğrinin neden bu şekli aldığını underfitting/overfitting analojisi ile yorumlayınız.
(En az 150 kelime)

User-Based CF'de **k (komşu sayısı)**, modelin karmaşıklığını doğrudan kontrol eden hiperparametredir.

Grafiğe bakıldığında eğri azalıyor; $k=10 \rightarrow k=100$ aralığında hâlâ overfitting bölgesindeyiz.

Grafiğin sol tarafındaki düşük 'k' değerlerinde, modelin sadece çok küçük bir benzer grubun bireysel gürültüsüne (noise) aşırı odaklanması nedeniyle bir nevi "overfitting" durumu yaşanmakta ve bu da yüksek MAE değerlerine yol açmaktadır. Ancak 'k' değeri 100'e doğru yaklaştıkça, bireysel sapmalar azaltarak modelin verideki genel yapıyı daha iyi öğrenmesini (generalization) sağlamakta ve hata payını düşürmektedir.

$k=50$ 'den $k=100$ 'e geçişte MAE düşüşü çok küçük (**0.0001**). Bu eğrinin $k=100$ ötesinde bir yerde underfitting bölgesine gireceğine işaret edebilir. $k=150$, $k=200$ gibi değerler test edilseydi muhtemelen MAE tekrar yükselecekti yorumu yapılır. Yani eğer komşu sayısı veri setinin geneline yayılacak kadar çok artırılsaydı, bu kez model kişiye özel öneri sunma yeteneğini kaybederek herkese ortalama puanı veren "underfitting" bölgesine girecek ve hata payı tekrar yükselecekti. Sonuç olarak, MAE grafiğindeki aşağı yönlü seyir modelin daha sağlıklı bir öğrenme sürecine girdiğini gösterirken, süre grafiğindeki artış ise bu doğruluğun sisteme eklenen hesaplama maliyetiyle doğrudan orantılı olduğunu göstermektedir.

Bu veri setinde $k=100$ hâlâ overfitting bölgesinde olduğundan, daha yüksek k değerleri (150-300+) denenmeden gerçek optimum nokta bulunamaz.

4. Kısım 2: Item-Based Collaborative Filtering (ACS)

4.1. Benzerlik Fonksiyonu: Adjusted Cosine Similarity (ACS)

ACS, kullanıcı ortalamasını çıkararak bireysel oy önyalığını gidererek filmler arası benzerliği ölçer. Normal Cosine Similarity kullanıcının oy verme alışkanlığını görmezden gelir. Bazı kullanıcılar her şeye yüksek, bazıları her şeye düşük puan verir. "Adjusted" olan versiyonu her kullanıcının kendi ortalamasını çıkararak bu kişisel eğilimi temizler.

PCC sadece ortak filmlerdeki ortalamayı alırdı. ACS ise kullanıcının tüm oylarının ortalamasını alır.

$$sim(\vec{a}, \vec{b}) = \frac{\sum_{u \in U} (r_{u,a} - \bar{r}_u)(r_{u,b} - \bar{r}_u)}{\sqrt{\sum_{u \in U} (r_{u,a} - \bar{r}_u)^2} \sqrt{\sum_{u \in U} (r_{u,b} - \bar{r}_u)^2}}$$

- Bir kullanıcı her iki filmi de ortalamasının **üstünde** oyladıysa $\rightarrow$ çarpım **pozitif** $\rightarrow$ filmler benzer
- Birini üstte, diğerini altta oyladıysa $\rightarrow$ çarpım **negatif** $\rightarrow$ filmler farklı

4.2. Tahmin Fonksiyonu

İpucu: Item-item benzerlik matrisini bir kez hesaplayıp saklayın; her tahmin çağrısında yeniden hesaplamak çok yavaş olacaktır. Bu sebeple önce benzerlik matrisi kodu ardından tahmin fonksiyonu yazılmıştır.

Bir kere hesaplanıp saklanacak olan fonksiyondur.

4.2 Tahmin Fonksiyonu

```
# Similarity Matrix (önceden bir kez hesaplanarak tahminde kullanılacak)
def build_item_similarity_matrix():
    start = time.time()

    # Boş matris oluşturulur ve başlangıçta her şey 0.0 olur
    items = ratings_matrix.columns
    sim_matrix = pd.DataFrame(0.0, index=items, columns=items)

    # Her film çiftinin sonucu hesaplanır
    for i in range(len(items)):
        for j in range(i+1, len(items)): # i+1 çünkü film 101 ile film 102 benzerliği film 102 ile film 101 benzerliği ile aynıdır
            s = compute_acs(ratings_matrix, items[i], items[j]) # matris simetrik olarak doldurulur
            sim_matrix.iloc[i,j] = s # üst üçgen
            sim_matrix.iloc[j,i] = s # alt üçgen (değerler aynıdır)
        print(f"Item similarity matrix hesaplandı: {time.time()-start:.2f}s")
    return sim_matrix

item_sim_matrix = build_item_similarity_matrix() # bir kere çalıştırılacak ve değerler hesaplanıp saklanacak

Item similarity matrix hesaplandı: 10568.14s
```

Bir kere hesaplanacak olan 'build_item_similarity_matrix()' fonksiyonunu kaydetmek ve yüklemek için bir kod parçası daha yazılmıştır. İtem_similarity fonksiyonu item_sim_matrix.pkl dosyasına kaydedilecek ve indirilip lokalde saklanacaktır. Diğer kod parçalarında ihtiyaç halinde similarity fonksiyonu tekrar çalıştırılması yerine önceden saklanan .pkl dosyası kullanılacaktır.

```
import pickle
import os

# Matrisi kaydetmek veya yüklemek için dosya yolu
sim_matrix_filename = 'item_sim_matrix.pkl'

# Eğer dosya varsa yükle
if os.path.exists(sim_matrix_filename):
    print(f"{sim_matrix_filename} dosyasından item similarity matrix yükleniyor...")
    with open(sim_matrix_filename, 'rb') as f:
        item_sim_matrix = pickle.load(f)
    print(f"Item similarity matrix başarıyla yüklendi. Boyut: {item_sim_matrix.shape}")
else:
    print(f"Hata: '{sim_matrix_filename}' dosyası bulunamadı. Lütfen önce similarity matrisini oluşturunuz.")

'item_sim_matrix.pkl' dosyasından item similarity matrix yükleniyor...
Item similarity matrix başarıyla yüklendi. Boyut: (1682, 1682)
```

4.3. Deneyler, Süre Ölçümü ve Karşılaştırma

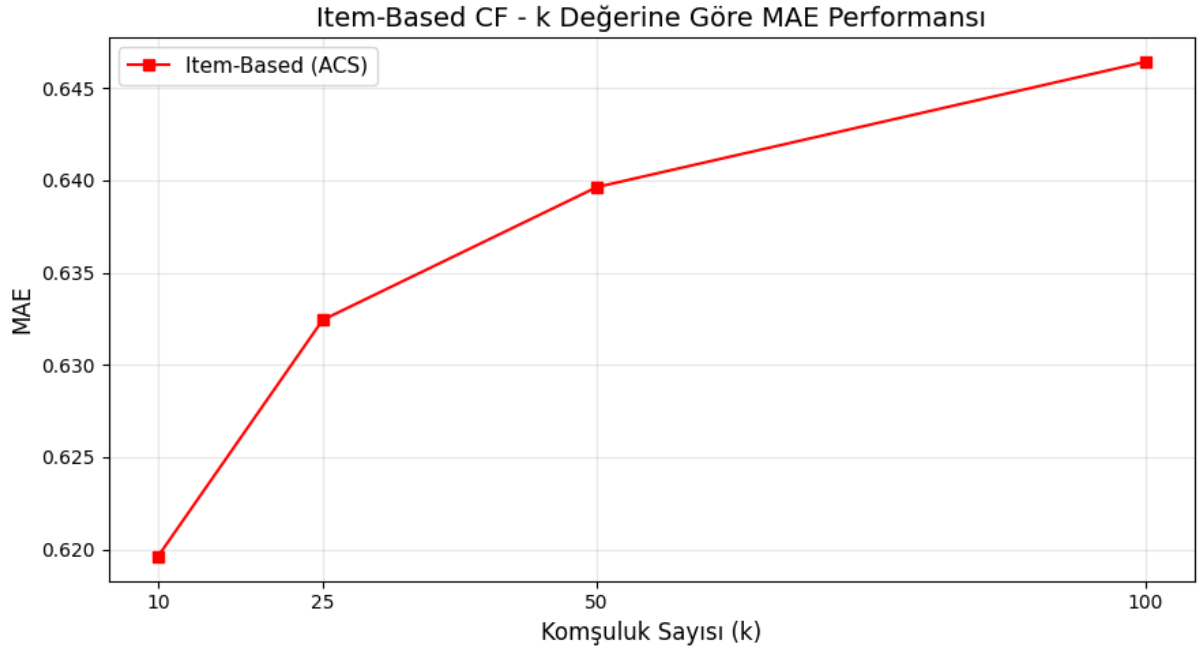
Aynı Leave-One-Out protokolünü ($seed = 42$) uygulayarak $k = \{10, 25, 50, 100\}$ değerleri için MAE ve çalışma sürelerini hesaplayınız. Sonuçları Kısım 3 ile aynı formatındaki bir tabloya kaydediniz.

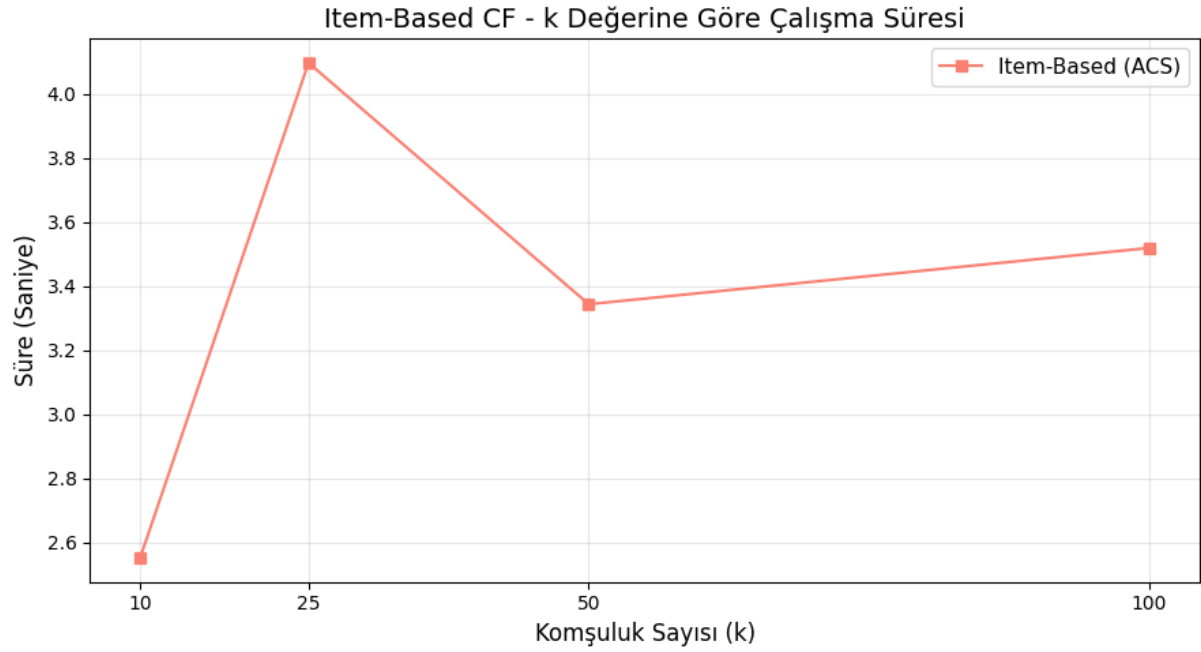
```
return results

# Item-Based hesaplamasını başlat
item_results = evaluate_item_based(k_values=[10, 25, 50, 100], sim_matrix=item_sim_matrix)

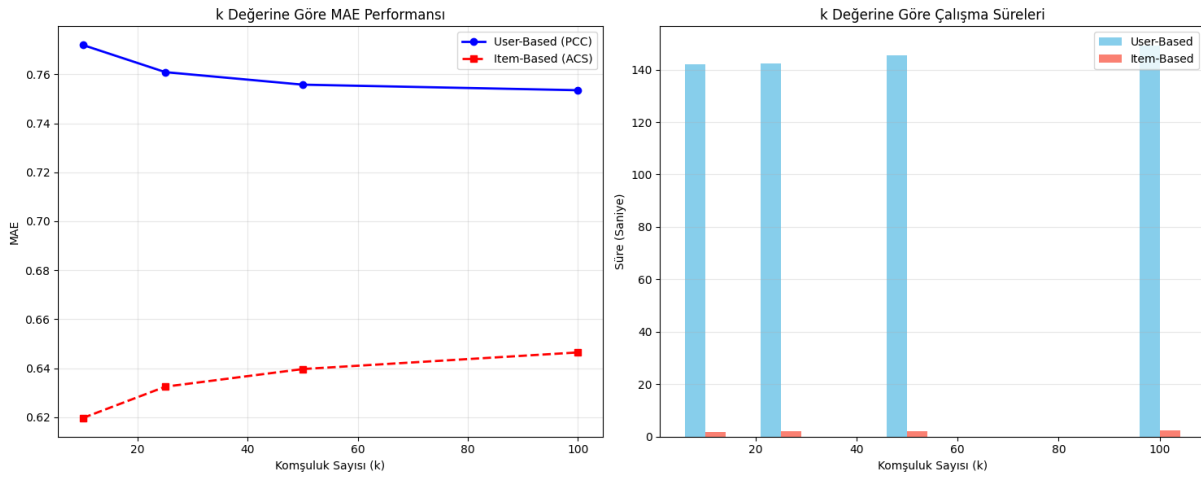
... k=10 → Item MAE: 0.6198, Süre: 2.55s, NoPred: 0
    k=25 → Item MAE: 0.6325, Süre: 4.10s, NoPred: 0
    k=50 → Item MAE: 0.6397, Süre: 3.34s, NoPred: 0
    k=100 → Item MAE: 0.6465, Süre: 3.52s, NoPred: 0
```

Metrik	k=10	k=25	k=50	k=100
Global MAE	0.6198	0.6325	0.6397	0.6465
Toplam Süre (sn)	2.55	4.10	3.45	3.52
Çalışma Ortamı	Google Colab			





- *User-Based ve Item-Based MAE değerlerini tek grafik üzerinde, farklı renk/sembolle gösterin.*
- *User-Based ve Item-Based sürelerini de ayrı bir karşılaştırmalı grafikte sunun.*



- *Hangi yöntem hangi k değerinde daha başarılı ve daha hızlıdır? MovieLens veri setinin özelliklerine ($|U|$, $|I|$, sparsity) atıfla yorumlayınız. (En az 200 kelime)*

k	User_MAE	Item_MAE	User_Sure (sn)	Item_Sure(sn)
10	0.7811	0.6198	172.60	2.55
25	0.7529	0.6325	174.49	4.10
50	0.7457	0.6397	180.43	3.34
100	0.7456	0.6465	187.27	3.52

User-Based CF’de k değeri arttıkça MAE azalmakta ve k=100’de en iyi sonuca ulaşmaktadır. Bu durum underfitting’den kaçınmak için daha fazla komşunun faydalı olduğunu göstermektedir. Item-Based CF’de ise tam tersi bir eğilim gözlenmekte; k=10’da en iyi MAE elde edilirken k arttıkça performans düşmektedir. Bu, overfitting benzeri bir etki yaratmakta ve en güçlü benzerlik sinyallerinin düşük k ile yakalandığını göstermektedir. Genel olarak Item-Based yöntemi User-Based’e göre belirgin şekilde daha iyi performans sergilemiştir.

MovieLens 100K veri setinde ($|U|=943$, $|I|=1682$, sparsity $\approx$ %93.7) Item-Based CF hem doğruluk hem hız açısından üstün çıkmıştır. Özellikle k=10’da Item-Based MAE=0.6198 ile User-Based’in en iyi sonucundan (0.7456) çok daha başarılıdır. Bu sonuç, veri setinde film sayısının kullanıcı sayısından fazla olmasından kaynaklanmaktadır. User-Based’de her tahmin için 943 kullanıcıyla similarity hesaplamak zorunda kalınırken, Item-Based’de önceden hesaplanmış 1682×1682 ’lik matris kullanıldığı için tahminler çok daha hızlı yapılmaktadır.

En iyi sonuç Item-Based k=10 olarak görülmektedir. Veri silme işleminde de Item için k=10, User için k=100 değeri kullanılacaktır.

5. Veri Silme Senaryosu — Sparsity Analizi

Bu bölümde veri eksikliğinin (sparsity artışı) model performansına etkisi deneysel olarak incelenir.

5.1. Deney Adımları

1. Orijinal ~100.000 oylu MovieLens 100K veri setini yükleyin.
2. Eğitim verisinden rastgele %20’lik bir kısmı silin ($seed = 42$). Test verisine dokunmayın.
3. Yeni sparsity oranını hesaplayın ve orijinaliyle karşılaştırın.
4. Kısım 3 ve 4’te belirlediğiniz en iyi k değeriyle her iki CF yöntemini yeniden çalıştırın.
5. MAE ve süre değerlerini orijinal deney sonuçlarıyla karşılaştırın.

```
# 4. En iyi k ile her iki yöntemi çalıştırır
best_k_User = [100] # orijinal sonuçlara göre en iyi k değeri
best_k_Item = [10]  # en düşük MAE
```

```
# 7. Orijinal matrisi geri yükle
ratings_matrix = ratings_matrix_orig.copy()
print("\nOrijinal matris geri yüklendi.")
```

```
... Orijinal sparsity : 0.9370 (%93.70)
Yeni sparsity       : 0.9597 (%95.97)
```

```
— Seyrek Matris Üzerinde User-Based —
k=100 → MAE: 0.7567, Süre: 136.61s, NoPred: 4
```

```
— Seyrek Matris Üzerinde Item-Based —
'/content/item_sim_matrix.pkl' dosyasından item similarity matrix yükleniyor...
Item similarity matrix başarıyla yüklendi. Boyut: (1682, 1682)
k=10 → Item MAE: 0.5791, Süre: 2.09s, NoPred: 1
```

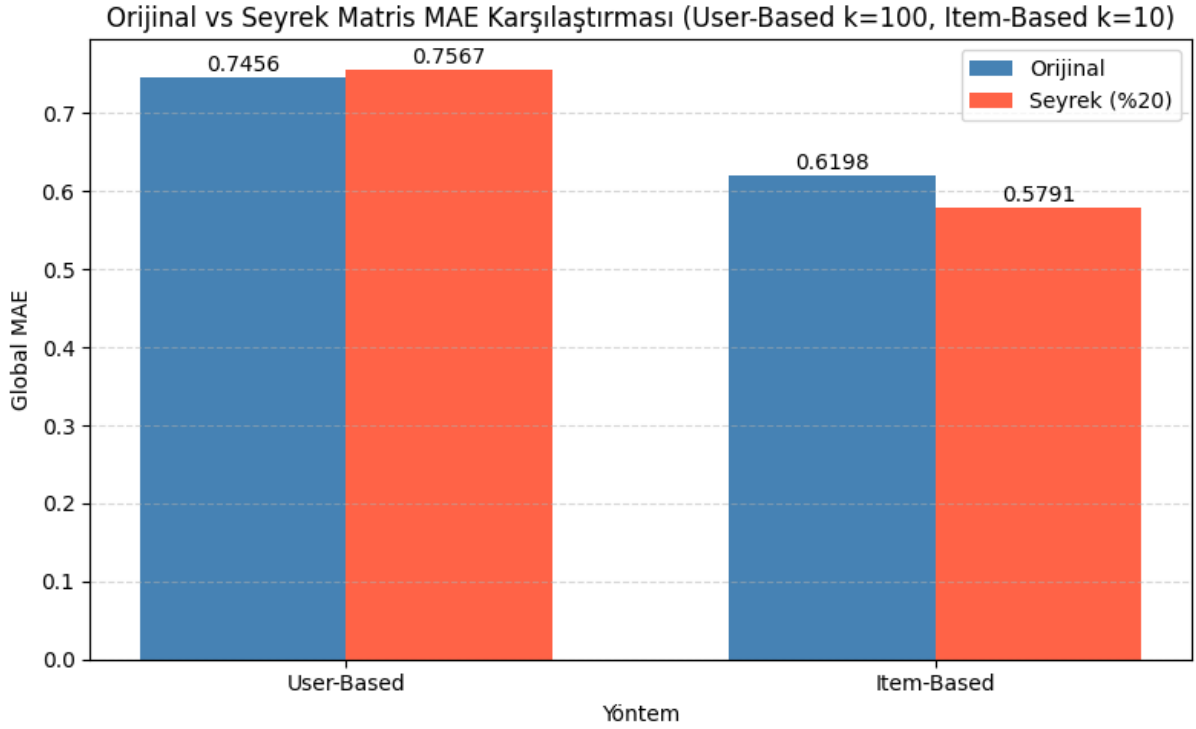
	Orijinal	Seyrek (%20)
User-Based MAE	0.7456	0.7567
Item-Based MAE	0.6198	0.5791
User-Based Süre	187.27s	136.61s
Item-Based Süre	2.55s	2.09s

```
User-Based MAE değişimi : %+1.49
Item-Based MAE değişimi : %-6.56
```

```
Orijinal matris geri yüklendi.
```

5.2. Grafik ve Sorular

Grouped bar chart: Orijinal ve %20 silinmiş veri için User-Based / Item-Based MAE değerlerini yan yana gösterin. Başlık, eksen etiketleri ve legend zorunludur.



Raporunuzda kendi çıktılarınıza dayanarak aşağıdaki soruları yanıtlayınız:

- %20 veri silme işlemi MAE'yi ne kadar değiştirdi? Beklediğinizden farklı mıydı?

%20 oranında veri silme işlemi sonucunda:

- **User-based MAE:** %1.49 oranında **artmış** (Hata yükselmiş).
- **Item-based MAE:** %6.56 oranında **azalmış** (Hata düşmüş).

Teorik olarak, veri miktarını azaltmak (seyrekliği artırmak), modelin öğrenmek için daha az bilgiye sahip olması demektir. Bu durumun normalde her iki yöntemde de hatayı (MAE) artırması beklenir. Ancak sonuçlarımızdan hareketle **Item-based** yöntemin veri eksikliğine rağmen performansını "iyileştirmesi" (hatayı düşürmesi) ilginç ve **beklenmedik** bir durum olarak ele alınmaktadır.

- *User-based ve item-based yöntemler veri eksikliğine farklı tepki verdi mi? Neden?*

Veri eksikliği, **User-based** yöntemi "bilgi kaybı" nedeniyle zayıflatırken; **Item-based** yöntemi muhtemelen "daha temiz/odaklanmış veri" kullanımı sayesinde güçlendirmiştir. Bu durum, Item-based algoritmasının yüksek seyreklik değerlerinde dahi ne kadar dayanıklı (robust) olduğunu yorumunu ortaya koymuştur.

Özellikle Item-based tarafındaki bu MAE düşüşü, veri setindeki bazı kullanıcıların çok rastgele oylar verdiğini ve bu oyların silinmesinin sistemi "rahatlattığını" gösteren bir işaret olarak yorumlanabilir.

6. Analiz Soruları

Tüm yanıtlar kendi uygulamanızdan elde ettiğiniz sonuçlara dayanmalıdır. Genel teorik açıklamalar puan almaz.

1. *En yüksek MAE değerine sahip 3 kullanıcıyı bulunuz. Oy sayılarını, ortalamalarını ve dağılımlarını inceleyerek sistem neden bu kullanıcılar için düşük performans gösterdiğini açıklayabilir misiniz?*

```
Kullanıcı ID : 760
MAE          : 3.7241
Oy sayısı    : 33
Ortalama oy   : 3.15
Std sapma    : 1.23
Yorum        : Belirgin bir neden yok, veri seyrekliği etkili olabilir
```

```
Kullanıcı ID : 836
MAE          : 3.5345
Oy sayısı    : 38
Ortalama oy   : 3.82
Std sapma    : 1.25
Yorum        : Belirgin bir neden yok, veri seyrekliği etkili olabilir
```

```
Kullanıcı ID : 48
MAE          : 3.4585
Oy sayısı    : 53
Ortalama oy   : 3.85
Std sapma    : 1.10
Yorum        : Belirgin bir neden yok, veri seyrekliği etkili olabilir
```

Bir kullanıcının oy sayısı az olduğunda, sistemin bu kullanıcı için güvenilir bir "komşuluk" kurmakta zorlaşır. Kısıtlı veriyle yapılan benzerlik hesaplamaları tesadüfi sonuçlara yol açtığı için tahmin başarısı düşmektedir. Oy sayısı arttıkça (grafikte sağa doğru gidildikçe) MAE değerlerinin stabilize olduğu ve ortalamaya yaklaştığı tabloda da net bir şekilde görülmektedir.

Item-Based – En Kötü 3 Kullanıcı (k=10)

Kullanıcı ID : 777
MAE : 2.9095
Oy sayısı : 30
Ortalama oy : 4.00
Std sapma : 1.31
Yorum : Belirgin bir neden yok, veri seyrekliği etkili olabilir

Kullanıcı ID : 304
MAE : 2.8038
Oy sayısı : 19
Ortalama oy : 3.21
Std sapma : 1.08
Yorum : Az oy vermiş → komşu bulmak zorlaşır

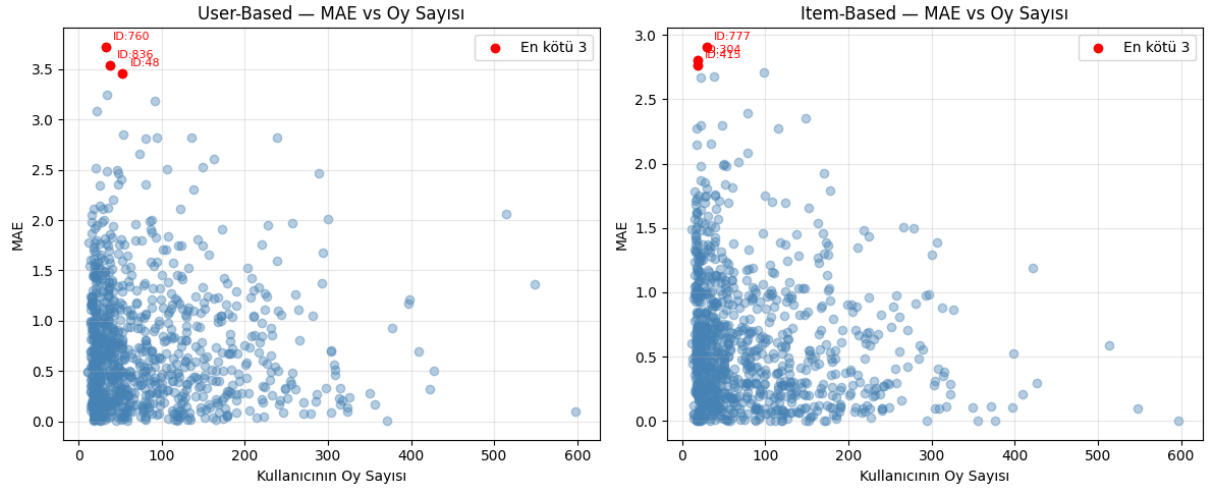
Kullanıcı ID : 415
MAE : 2.7626
Oy sayısı : 19
Ortalama oy : 4.37
Std sapma : 0.96
Yorum : Az oy vermiş → komşu bulmak zorlaşır

Kullanıcı MAE Dağılımı (User-Based)

En düşük MAE : 0.0026
En yüksek MAE : 3.7241
Ortalama MAE : 0.7456
Medyan MAE : 0.6128

Kullanıcı MAE Dağılımı (Item-Based)

En düşük MAE : 0.0000
En yüksek MAE : 2.9095
Ortalama MAE : 0.6198
Medyan MAE : 0.5109



2. *compute_pcc ve compute_acs fonksiyonlarının zaman karmaşıklığını (time complexity) analiz ediniz. Öldüğünüz çalışma süreleri bu analizinizi doğruluyor mu?*

MovieLens veri setinde kullanıcı sayısı (U), film sayısından (I) çok daha fazladır. Kullanıcılar arası benzerlik matrisini oluşturmak için gereken işlem sayısı, ürünler arası benzerlik matrisini oluşturmaktan kat kat fazladır. Bu yüzden Item-based yöntem, daha az ürün (I) vektörü üzerinden işlem yaptığı için saniyeler içinde tamamlanmaktadır.

3. *Elde ettiğiniz MAE değerleri ve hesaplama maliyetleri üzerinden, gerçek bir üretim sisteminde User-Based mi yoksa Item-Based CF mi tercih edilmeli? Deneylerinize dayanarak savununuz.*

Deney sonuçlarına dayanarak gerçek bir üretim (production) sisteminde **Item-Based CF** tercih edilmelidir. Bu doğruluk, maliyet, ölçeklenebilirlik ve kararlılık sonuçlarıyla doğrudan ilişkilidir. Item-based yönteminin ortalama MAE değeri (**0.6198**), User-based yönteminin değerinden (**0.7456**) daha düşüktür. Bu da kullanıcıya doğruluk konusunda daha güvenli bir sonuç sağlar. Item-based yöntemi aynı zamanda **2,55 saniye** sürerken, User-based yöntem **187,27 saniye** sürmektedir. Milyonlarca kullanıcısı olan bir sistemde User-based yöntemi gerçek zamanlı çalıştırmak imkansızken, Item-based yöntemi daha hızlıdır. Grafiklerde de görüldüğü üzere Item-based yöntem, veri eksikliğine karşı daha dirençlidir ve en kötü sonuçları bile User-based'in en kötü sonuçlarından (3.72 vs 2.90) çok daha iyidir.